COMPUTER NETWORKS · SYSTEM DESIGN

Remote System Health Monitoring

A lightweight path from local readings to protected alerts (Deliverables 1 & 2)

Small agents send local health updates, the server monitors them, and SDN dynamically throttles congestion and prioritizes critical alerts.

Team SRN:

PES2UG25CS609

PES2UG25CS620

PES2UG25CS642

HOW IT WORKS

Each node sends a small update—no permanent connection needed

Every node sends a small UDP update on a timer, so the monitor does not need a separate TCP connection for each client.

01

Collect local data

psutil gathers CPU and memory use, process counts, top processes, and network I/O.

02

Keep each report small

Strict JSON includes the node ID, sequence, timestamp, interval, priority, and metrics.

03

Prioritize alerts

DSCP 46 marks threshold violations, allowing OpenFlow to route them to Queue 2.

PACKET RULES · COMMON/PROTOCOL.PY

```
payload=json.dumps(report,separators=(",",":")).encode(
)
if len(payload)>1400: raise ProtocolError("payload
exceeds MTU guard")
validate(report)
```

VALIDATION AND THE ≤1400-BYTE GUARD KEEP EACH DATAGRAM COMPACT AND PREVENT FRAGMENTATION.

WHAT THE SERVER SAVES · SERVER_REPORTS.CSV

```
node_id,seq,delay_ms,priority
h_partial,1,0.24,NORMAL
h_partial,10,0.26,NORMAL
```

THE SERVER STORES EVIDENCE FOR EVERY PACKET, NOT JUST A LIVE DASHBOARD.

AGENT DESIGN · LOCAL SAMPLING

How small agents turn local readings into useful telemetry

A steady timer prevents drift, and repeatable simulation modes make alerts and failures easy to test.

What each report includes
node_id · seq · sent_ts · interval · priority · validated metrics

What happens when collection fails
HEARTBEAT still goes out, so the node stays alive even when a metric read fails.

Message size
HEALTH: 378-byte payload / 420-byte wire size / 3.36 kbps at 1 s.

DSCP / TOS MARKING

```
# DSCP 46 (EF); 46 << 2 = 184
sock.setsockopt(socket.IPPROTO_IP,
                socket.IP_TOS, 0xB8)
```

Normal reports use DSCP 0; threshold violations use EF marking.

STATS_COLLECTOR.PY · MODES

```
if pattern == "spike":
    cpu, processes = 95.0, 550
elif pattern == "normal":
    cpu, processes = safe_values()
```

In spike mode, CPU reaches at least 95% and processes reach 550 on configured ticks. Normal mode stays below warning thresholds.

SERVER DECISION LOOP · RECEIVE, CHECK, ALERT

The server turns each packet into a decision

A non-blocking UDP socket and `select` handle telemetry without creating a thread for every client.

SELECT LOOP + DELAY

```
select.select([sock], [], [],
0.5)
sock.recvfrom(2048)
decode_and_validate(data)
Delay: (time.time() -
report["sent_ts"]) * 1000
```

If the clock jumps, the delay is clamped to 0.0 ms and a warning is recorded.

REPORTS AND ALERTS STAY APPEND-ONLY, GIVING THE DASHBOARD AND EXPERIMENTS A RELIABLE RECORD.

OUTPUT · CSV EVIDENCE

Recorded decisions
server_reports.csv: h_partial,1,0.24 ms,NORMAL
server_reports.csv: h_partial,10,0.26 ms,NORMAL
server_alerts.csv: node_silence,3.0,CRITICAL,0.0 ms
server_alerts.csv: packet_loss_percent,20.0,CRITICAL,0.26 ms

ALERT THRESHOLDS

CPU: warn at 70%, critical at 90%; memory: warn at 75%, critical at 90%; processes: warn at 300, critical at 500; packet loss: warn at 5%, critical at 20%.

Static OpenFlow rules reserve dedicated switch queues

The agent marks critical telemetry with DSCP 46, and OpenFlow directs datagrams into guaranteed hardware queues over the 10 Mbps bottleneck.

OPENFLOW 1.3 · MATCH → ACTION

DSCP 46

At the socket, the agent uses EF marking (DSCP 46 / TOS 0xB8). OpenFlow 1.3 matches IPv4 (eth_type=0×0800), UDP (ip_proto=17), port 5005, and DSCP 46.

Priority 100 sends this traffic to Queue 2 with OFPActionSetQueue(2), then outputs to OFPP_NORMAL. Priority 50 maps DSCP 0 to Queue 1; priority 10 sends bulk traffic to Queue 0.

HTB RESERVATION · TOPOLOGY.PY

Reserved queue capacity

Queue	Traffic	Minimum
Queue 2	critical	5 Mbps
Queue 1	normal	3 Mbps
Queue 0	best-effort	1 Mbps

THESE MINIMUMS KEEP A CRITICAL LANE OPEN WHEN BACKGROUND TRAFFIC FILLS THE LINK.

DYNAMIC SDN CONTROL · REAL-TIME ADAPTIVE MITIGATION (D2)

SDN controller dynamically mitigates congestion & isolates rogue hosts

Ryu controller polls port statistics every 2.0s and dynamically injects reactive OpenFlow rules to preserve critical queue headroom.

PORT LOAD TELEMETRY & ADAPTIVE THROTTLING

OFPPortStats

Every 2.0s, the Ryu controller polls switch ports via OFPPortStatsRequest. When bottleneck TX load reaches >= 8.0 Mbps (80%), the controller dynamically issues an OFPFlowMod to demote bulk cross-traffic to Queue 0 with idle_timeout=15s.
When port rate drops below 4.0 Mbps, the controller issues OFPFC_DELETE_STRICT to decommission the throttle rule and restore fair sharing. If an untrusted host floods traffic, a Priority 200 DROP rule isolates it.

REACTIVE FLOW MODULATION · RYU_CONTROLLER.PY

Dynamic mitigation policies

Condition	Trigger / Threshold	Dynamic OpenFlow Rule
Bottleneck Congestion	TX rate >= 8.0 Mbps	Priority 80 → Queue 0 (idle=15s)
Bandwidth Restored	TX rate < 4.0 Mbps	OFPFC_DELETE_STRICT (Priority 80)
Rogue Flooder / DoS	Malformed packet burst	Priority 200 → DROP (actions=[])

DYNAMIC OPENFLOW MODULATION OPERATES AT WIRE SPEED WITHOUT MANUAL OPERATOR INTERVENTION.

When a quiet node becomes a visible failure

Sequence gaps detect packet loss, silence timeouts flag crashes in 3.0s, and the MTU guard safely drops 10,000 malformed packets.

SEQUENCE GAPS AND SILENCE

When reports stop, the last-seen time keeps advancing. Duplicates and out-of-order packets are tracked separately, while a reset to 0 or 1 triggers restart handling.

```
gap = seq - last_seq - 1
loss_rate = lost / (received + lost)
if now - last_seen >
    3.0 * interval:
    emit("NODE_DOWN")
```

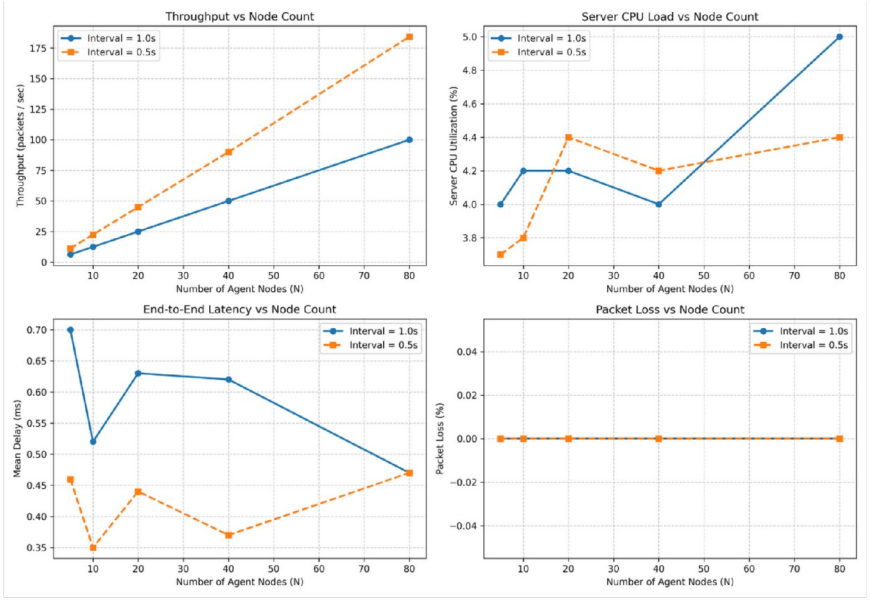
MEASURED FAILURE RESULTS

Failure	TTD
agent_crash	1.07 ± 0.04 s
link_failure	3.30 ± 0.00 s
partial_loss	3.31 ± 0.02 s
recovery on resume	< 0.05 s

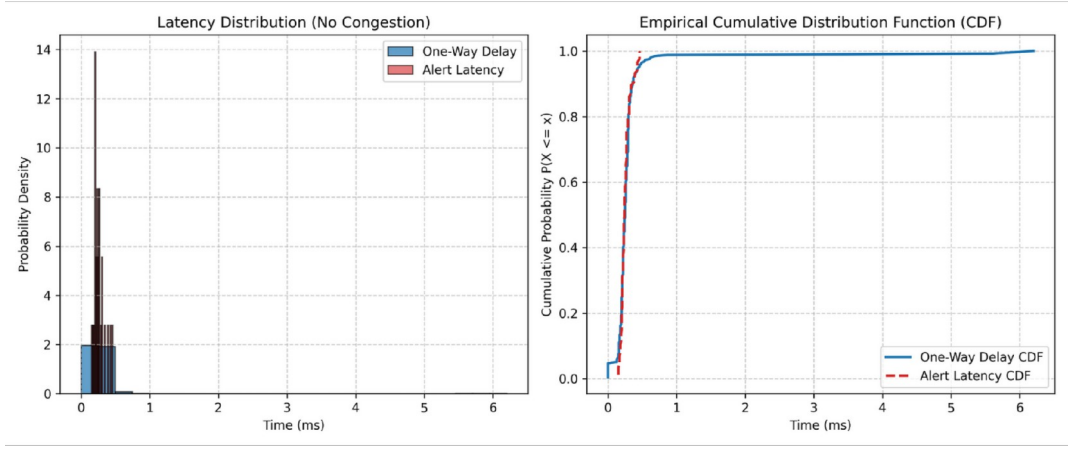
ROBUSTNESS VERIFIED ACROSS THREE SCENARIOS: 12 MBPS CONGESTION, NODE SILENCE CUTS, AND MALFORMED BURSTS.

The results: fast alerts that scale

Empirical benchmarks compare the proposed SDN QoS architecture against a standard baseline FIFO queue under severe congestion.



CONGESTION_COMPARISON.PNG · CRITICAL LOSS: 56.67% WITHOUT PRIORITY, 0.00% WITH PRIORITY; P95 DELAY: 30.75 MS VS 1.48 MS



LATENCY_DISTRIBUTION.PNG · ALERT LATENCY STAYS CONCENTRATED BELOW 1 MS IN THE SUPPLIED SUMMARY

Baseline performance

Alert latency: 0.257 ms mean; 0.430 ms at p95

Priority routing on

Critical loss: 0.00%; critical p95 delay: 1.48 ms

Scale

5–80 nodes; mean delay stays below 0.7 ms; CPU stays at or below 5.0%

What we built — and what we learned

Lightweight UDP reports become reliable through validation and sequence tracking. Dynamic SDN QoS guarantees critical health alerts survive severe congestion.

Start on your machine

```
python3 server/monitor_server.py --
port 5005
python3 agent/health_agent.py --id=h1
\
 --server 127.0.0.1 --port 5005 --
interval 1.0 --mode real
```

Run the tests

```
pytest tests/ -v --cov=common --cov=server --cov=agent --
cov=sdn
65 passed · Full verification across core & SDN controller
python3 demo.py  # Option 5: Deliverable 2 Showcase
```

What the results show

Critical-packet loss: 56.67% with Priority OFF vs 0.00% with Priority ON. Scale: up to 80 nodes, 184.2 packets/s, with server CPU at or below 5.0%. Dynamic SDN throttle activates at >= 8.0 Mbps.